7 Lecture 7: Value Function Approximation

Introduction

Introduction

在前面的 Lecture 中一个隐含的默认假设是 state value 与 action value 都是离散的方程，即对于每个离散的 s_i 和 (s_i, a_j) 有相对应的 state value $v_\pi(s_i)$ 和 action value $q_\pi(s_i, a_j)$. 因此比较适合用表格 (table) 的方式进行表示:

s_1	s_2	s_3	s_4	$\cdots$	s_9
$v_\pi(s_1)$	$v_\pi(s_2)$	$v_\pi(s_3)$	$v_\pi(s_4)$	$\cdots$	$v_\pi(s_9)$

Table 7.1: Table of state values

State / Action	a_1	a_2	a_3	a_4	a_5
s_1	$q_\pi(s_1, a_1)$	$q_\pi(s_1, a_2)$	$q_\pi(s_1, a_3)$	$q_\pi(s_1, a_4)$	$q_\pi(s_1, a_5)$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
s_9	$q_\pi(s_9, a_1)$	$q_\pi(s_9, a_2)$	$q_\pi(s_9, a_3)$	$q_\pi(s_9, a_4)$	$q_\pi(s_9, a_5)$

Table 7.2: Table of action values

使用表格形式表示数据的优点在于易于理解和分析，但缺点在于难以处理大型或连续的 state / action space:

1. **Difficulty 1: Storage for Discrete Space.** 对于离散空间，空间较大时离散值较多，因此需要大量存储空间.
2. **Difficulty 2: Storage for Continuous Space.** 对于连续空间，处理时需要进行(网格)离散化. 当网格稀疏时不能很好地近似原空间，当网格稠密即节点过多时又面临与 Difficulty 1 一样的存储问题.
3. **Difficulty 3: Generalization.** 使用表格进行更新只能逐点进行，单点操作不能够影响周围未被访问到的值.

7.1 Function Approximation

Example

使用表格表示数据需要的存储量一般与数据量 N 成正比 ($O(N)$)，一种解决方式是使用这些数据通过函数近似 (**function approximation**) 拟合 action / state value function，最终只需要存储相应函数的少量参数即可.

Example 7.1. 考虑一维 *state* $s_1, \ldots, s_{|\mathcal{S}|} \in \mathbb{R}$，相应的 *state value* 为 $v_\pi(s_1), \ldots, v_\pi(s_{|\mathcal{S}|})$，其中 $\mathcal{S}$ 为 *state space*, $|\mathcal{S}|$ 为所有的状态个数, π 为给定的 *policy*. 假设 $|\mathcal{S}|$ 非常大，最简单的方式是使用直线 (如图 7.1) 拟合这些 *state-state value pair*，即 $(s, v_\pi(s))$，这样只需要存储该直线的少量参数. 拟合后的 *state value function* 可以写成如下形式:

$$\hat{v}(s, w) = as + b = \underbrace{[s, 1]}_{\phi^T(s)} \underbrace{\begin{bmatrix} a \\ b \end{bmatrix}}_{w} = \phi^T(s) w \tag{7.1}$$

其中 $w \in \mathbb{R}^m$ 为 *parameter vector* (参数向量)，$\phi(s)$ 为 s 的 *feature vector* (特征向量).

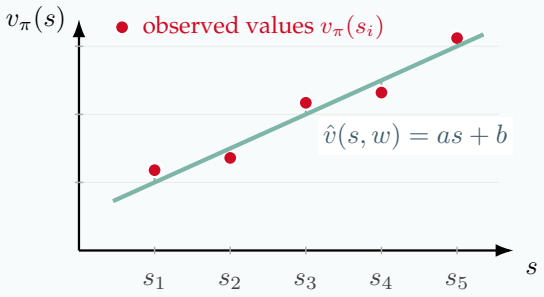


Figure 7.1: Linear approximation of a state-value function.

Curve Fitting

使用 function approximation 与 tabular method 的不同之处在于:

1. **Store:** tabular method 需要存储 $|\mathcal{S}|$ 个 state value，但 function approximation 只需要存储少量参数.
2. **Retrieve:** function approximation 中每当需要使用到 s 的 state value 时，需要重新计算 $\phi^T(s)w$.
3. **Accuracy:** 拟合/近似 (approximation) 会引入误差，因此我们牺牲了精度来提高存储效率.
4. **Update:** tabular method 逐点更新 state value 即可，而 function approximation 通过更新参数间接更新.
5. **Generalization ability:** function approximation 更新参数也会影响其他 state 的 state value. 因此只使用一个 state 的 experience sample 进行更新也可以帮助估计其他一些尚未访问 state 的 state value.

Note 7.1. 如果特征设计不好，由于 *function approximation* 有 *generalization ability*，其它状态可能错误地一起改变.

由于不同函数的表达能力不同，因此进行拟合前一个关键问题是选择什么样的拟合函数 $\hat{v}(s,w)$，主要有两种方式:

1. **Linear function:** $\hat{v}(s,w)=\phi(s)^Tw$，其中 $\phi(s)$ 为 feature vector，可以选取为多项式基底、Fourier 基底等. 更复杂的基底可以减小误差但相应地也增加了参数存储量，例如使用二阶曲线拟合:

$$\hat{v}(s,w)=as^2+bs+c=\underbrace{[s^2,s,1]}_{\phi^T(s)}\underbrace{\begin{bmatrix}a\\b\\c\end{bmatrix}}_{w}=\phi^T(s)w. \tag{7.2}$$

2. **Neural networks:** 特征向量 $\phi(\cdot)$ 的选择往往需要先验知识. 当没有先验知识时一个被广泛采用的解决方案是使用参数 θ 参数化的神经网络作为非线性函数逼近器输入神经网络的为 state，输出为非线性的 $\hat{v}(s,\theta)$.

Note 7.2. 在 *Linear function* $\hat{v}(s,w)=\phi(s)^Tw$ 中虽然 $\hat{v}(s,w)$ 对于 s 不一定是线性的，例如二阶曲线拟合 *(7.2)*. 但是对于参数 w 来说仍是线性的.

Tabular Method is One-hot Linear Function Approximation

虽然使用线性函数进行拟合的拟合能力不如选用非线性函数进行拟合，但线性情形下的理论分析较非线性情形时更简单、可以帮助理解非线性情形. 事实上 tabular method 实际上是线性函数逼近的一个特殊形式.

令 feature vector 为单位向量:

$$\phi(s)=e_s\in\mathbb{R}^{|S|}, \tag{7.3}$$

其中 e_s 是对应状态的 one-hot 向量，只有第 s 个元素为1，其余为0. 那么:

$$\hat{V}(s,w)=e_s^\top w=w(s). \tag{7.4}$$

其中 $w(s)$ 为 w 的第 s 个元素，就是表格中状态 s 对应的值. 因此:

Tabular Method = One-hot Linear Function Approximation.

7.2 Function Approximation based TD Learning

Introduction

现在，我们已经不再使用表格将所有的 state-state value pair 进行存储，而是基于 function approximation 方法使用参数化的 $\hat{v}(\cdot)$ 逼近 state value function $v_\pi(\cdot)$. 因此我们需要找到最优的参数向量 w 使 $\hat{v}$ 能够较好地近似 v_π.

这会产生两个重要问题:

① 如何度量 $\hat{v}(\cdot)$ 的好坏，或者说使用什么量作为标准去衡量 $\hat{v}(\cdot)$ 对 $v_\pi(\cdot)$ 的逼近程度.

② 如果有了该度量，那么应该朝着什么方向"努力"才能缩小 $\hat{v}(\cdot)$ 与 $v_\pi(\cdot)$ 的差异，以及如何设计相应的算法.

Objective Function

Objective Function. 度量 $\hat{v}(\cdot)$ 的好坏或 $\hat{v}(\cdot)$ 对 $v_\pi(\cdot)$ 的逼近程度一般使用目标函数 (objective function). 这里我们可以将目标函数定义为对于所有 state s，$\hat{v}(s)$ 与 $\hat{v}(x)$ 二者数值差异平方和的期望:

$$J(w) = \mathbb{E}[(v_\pi(S) - \hat{v}(S,w))^2]. \tag{7.5}$$

由于 $J(w)$ 变小意味着 $v_\pi(s)$ 与 $\hat{v}(s,w)$ 的差距变小，因此我们的目标是最小化 $J(w)$ 以找到 最优的 (optimal) w.

Note 7.3. 在深度学习中，目标函数也常被称为损失函数 *(loss function)*.

注意这里的 S 是一个随机变量，取期望时需要选择一定的分布进行计算. 分布的选择主要有如下两种方式:

① **Uniform distribution.** 即取到每个 state 的概率是一样的，都是 $1/|\mathcal{S}|$，此时目标函数实际可写为:

$$J(w) = \mathbb{E}[(v_\pi(S) - \hat{v}(S,w))^2] = \frac{1}{|\mathcal{S}|}\sum_{s\in\mathcal{S}}(v_\pi(s) - \hat{v}(s,w))^2. \tag{7.6}$$

平均分布假设所有的 state 同等重要，但实际上接近目标的状态应该是更重要的，需要分配更多的权重. 同时有一些 state 很少被访问，它们应该被分配更少的权重.

② **Stationary / steady-state / limiting distribution.** 描述了马尔可夫决策过程的长期行为 (long-run behavior)，即在给定 policy 下不断环境交互，达到稳定状态后每个 state 被访问概率的分布，将其记为 $\{d_\pi(s)\}_{s\in\mathcal{S}}$，其中 $d_\pi(s) \geqslant 0, \sum_{s\in\mathcal{S}} d_\pi(s) = 1$. 这样目标函数实际可写为:

$$J(w) = \mathbb{E}[(v_\pi(S) - \hat{v}(S,w))^2] = \sum_{s\in\mathcal{S}} d_\pi(s)(v_\pi(s) - \hat{v}(s,w))^2. \tag{7.7}$$

然而分布 $d_\pi(s)$ 的获取并不简单，一种方式是通过仿真实验近似:

$$d_\pi(s) \approx \frac{n_\pi(s)}{\sum_{s'\in\mathcal{S}} n_\pi(s')}. \tag{7.8}$$

实际上不需要进行仿真实验也可以根据 state 概率转移矩阵 P_π得到 $d_\pi(s)$ 的理论值，详细分析见附录 **??**.

Optimization: Gradient Descent

为最小化目标函数 $J(w)$，可以使用梯度下降法 (gradient descent, GD):

$$w_{k+1} = w_k - \alpha_k \nabla_w J(w_k), \tag{7.9}$$

其中梯度为:

$$\nabla_w J(w) = \nabla_w \mathbb{E}[(v_\pi(S) - \hat{v}(S,w))^2] = \mathbb{E}[\nabla_w (v_\pi(S) - \hat{v}(S,w))^2] = -2\mathbb{E}[(v_\pi(S) - \hat{v}(S,w))\nabla_w \hat{v}(S,w)] \tag{7.10}$$

由此得到相应的梯度下降 (GD) 和随机梯度下降 (SGD) 算法分别为:

$$\begin{aligned} &\text{GD:} \quad w_{k+1} = w_k + 2\alpha_k \mathbb{E}[(v_\pi(S) - \hat{v}(S,w_k))\nabla_w \hat{v}(S,w_k)], \\ &\text{SGD:} \quad w_{t+1} = w_t + 2\alpha_t \left(v_\pi(s_t) - \hat{v}(s_t,w_t)\right) \nabla_w \hat{v}(s_t,w_t). \end{aligned} \tag{7.11}$$

可以看到在进行梯度下降时梯度中已经包含了需要求解的目标 v_π，因此需要将其替换，常见的方法有两种:

1. **Monte Carlo:** 使用从 s_t 出发的一个 episode 的 discounted return g_t 代替 $v_\pi(s_t)$:

$$w_{t+1} = w_t + \alpha_t(g_t - \hat{v}(s_t,w_t))\nabla_w \hat{v}(s_t,w_t).$$

2. **TD learning:** 使用 $r_{t+1} + \gamma\hat{v}(s_{t+1},w_t)$ 代替 $v_\pi(s_t)$:

$$w_{t+1} = w_t + \alpha_t \left[r_{t+1} + \gamma\hat{v}(s_{t+1},w_t) - \hat{v}(s_t,w_t)\right] \nabla_w \hat{v}(s_t,w_t). \tag{7.12}$$

Note 7.4. 由于 α_t 可以自己设定，因此这里将原先 $2\alpha_t$ 的系数 2 都放入 α_t 中不写.

TD Learning with Function Approximation

当使用 $r_{t+1}+\gamma\hat{v}(s_{t+1},w_t)$ 代替 $v_\pi(s_t)$ 并使用线性函数 $\phi^T(s)w$ 逼近 $\hat{v}(s,w)$ 时，可以得到 TD-Linear 算法:

$$\text{TD linear: } w_{t+1}=w_t+\alpha_t\left[r_{t+1}+\gamma\phi^T(s_{t+1})w_t-\phi^T(s_t)w_t\right]\phi(s_t). \tag{7.13}$$

Algorithm 10 TD Learning with Function Approximation

1: **Initialization:** A function $\hat{v}(s,w)$ that is differentiable in w. Initial parameter w_0.
2: **Aim:** Approximate the true state values of a given policy π.
3: **for** each episode generated following the policy π **do**
4: **for** each step (s_t,r_{t+1},s_{t+1}) **do**
5: **In the general case:**

$$w_{t+1}=w_t+\alpha_t\left[r_{t+1}+\gamma\hat{v}(s_{t+1},w_t)-\hat{v}(s_t,w_t)\right]\nabla_w\hat{v}(s_t,w_t)$$

6: **In the linear case(TD linear):**

$$w_{t+1}=w_t+\alpha_t\left[r_{t+1}+\gamma\phi^T(s_{t+1})w_t-\phi^T(s_t)w_t\right]\phi(s_t)$$

7: **end for**
8: **end for**

7.3 Theoretical Analysis for TD-Linear

Convergence Analysis

由于非线性 approximation 的分析较为复杂，因此本部分仅考虑线性情形，即 TD-Linear (7.13).
在分析 TD learning 算法(7.13)之前，首先分析其确定性形式 (即 GD 算法):

$$w_{t+1}=w_t+\alpha_t\mathbb{E}\left[\left(r_{t+1}+\gamma\phi^T(s_{t+1})w_t-\phi^T(s_t)w_t\right)\phi(s_t)\right], \tag{7.14}$$

其中 s_t 的分布为 stationary distribution d_π. 分析确定性形式基于以下两点考虑:

1. 确定性算法的收敛性更容易 (尽管不是平凡的) 分析.
2. 确定性算法 (7.14) 的收敛性蕴含随机 TD 算法(7.13) 的收敛性.

为方便分析，首先定义如下两个矩阵:

$$\Phi=\begin{bmatrix}\vdots\\ \phi^T(s)\\ \vdots\end{bmatrix}\in\mathbb{R}^{n\times m},\quad D=\begin{bmatrix}\ddots & & \\ & d_\pi(s) & \\ & & \ddots\end{bmatrix}\in\mathbb{R}^{n\times n}, \tag{7.15}$$

其中 Φ 为特征向量矩阵，包含了 n 个 state 的特征向量 $\phi(s)\in\mathbb{R}^m$，D 为对角阵，元素 d_{ii} 为 state s_i 的 stationary distribution $d_\pi(s)$.

Lemma 7.1. 式*(7.14)*中的期望可以写为

$$\mathbb{E}\left[\left(r_{t+1}+\gamma\phi^T\left(s_{t+1}\right)w_t-\phi^T\left(s_t\right)w_t\right)\phi\left(s_t\right)\right]=b-Aw_t, \tag{7.16}$$

其中 $A\triangleq\Phi^TD\left(I-\gamma P_\pi\right)\Phi\in\mathbb{R}^{m\times m}$, $b\triangleq\Phi^TDr_\pi\in\mathbb{R}^m$.

Lemma 7.1 的证明可见附录**??**. 这样算法(7.14) 就可以写为:

$$w_{t+1}=w_t+\alpha_t(b-Aw_t). \tag{7.17}$$

Convergence target: 在分析收敛性之前我们首先需要找到收敛的目标，因此首先假设 $\exists\, w^*$，使得 $w_t\xrightarrow{t\to\infty}w^*$，因此 $w^*=w^*+\alpha_\infty(b-Aw^*)$，从而得到

$$w^*=A^{-1}b. \tag{7.18}$$

为保证式 (7.18)是有意义的，有几点说明：

1. A 可逆. 事实上 A 不仅可逆 (invertible)，而且是正定 (positive definite)的，即 $x^TAx > 0, \forall x \in \mathbb{R}^{\dim A}, x \neq \mathbf{0}$. 证明详见附录**??**.
2. $w^* = A^{-1}b$ 实际上是最小化投影贝尔曼误差 (projected Bellman error) 的最优解，详细说明见下一小节.
3. 在 tabular method 情形下，当 dim $w = n = |\mathcal{S}|$，$\phi(s) = [0, \dots, 1, \dots, 0]^T$时，有

$$w^* = A^{-1}b = v_\pi. \tag{7.19}$$

说明要学习的参数向量实际上是真实 state value，与表格 TD算法是TD-Linear算法的特例这一事实是一致的. 式 (7.19)的证明见附录**??**.

Convergence analysis: 在假设收敛的前提下得到收敛的目标 w^* 后，可以证明得到

$$w_t \xrightarrow{t\to\infty} w^* = A^{-1}b. \tag{7.20}$$

证明详见附录**??**.

Projected Bellman Error

事实上 TD-Linear 算法收敛的解 (7.19) 并不是 (7.5) 最小化的解，而是使投影贝尔曼误差 (**projected Bellman error**) 最小化的最优解，原因在于 ① 真实目标 v_π 未知 ② function approximation 带来的差异. 下面进行分析:

Objective Function 1: True value error. 目标函数构造的实际目标是使得 $\hat{v}$ 与真实的 v_π 尽可能接近:

$$J_E(w) = \mathbb{E}[(v_\pi(S) - \hat{v}(S, w))^2] = \|\hat{v}(w) - v_\pi\|_D^2, \tag{7.21}$$

其中 D 为 S 对应的分布，$\|x\|_D^2 = x^TDx = \|D^{1/2}x\|_2^2$.

Objective function 2: Bellman error. 由于 v_π 满足 Bellman equation $v_\pi = r_\pi + \gamma P_\pi v_\pi$，同时由于 v_π 未知，需要使用 $\hat{v}$ 替换. 所以期待 $\hat{v}(w)$ 尽可能满足方程 $\hat{v}(w) = r_\pi + \gamma P_\pi \hat{v}(w)$，代入式 (7.5) 中，得到:

$$J_{BE}(w) = \|\hat{v}(w) - (r_\pi + \gamma P_\pi \hat{v}(w))\|_D^2 \triangleq \|\hat{v}(w) - T_\pi(\hat{v}(w))\|_D^2 \tag{7.22}$$

其中 $T_\pi(x) \triangleq r_\pi + \gamma P_\pi x$，称为 Bellman operator. 最小化 Bellman error 是一个标准的最小二乘问题.

Objective function 3: Projected Bellman error. 由于使用的逼近函数可能出于函数结构原因无法完全满足 $\hat{v}(w) = r_\pi + \gamma P_\pi \hat{v}(w)$，因此可以使用投影矩阵 M 将 $T_\pi(\hat{v}(w))$ 投影到 $\hat{v}$ 的空间上使目标函数可以为 0:

$$J_{PBE}(w) = \|\hat{v}(w) - MT_\pi(\hat{v}(w))\|_D^2, \tag{7.23}$$

其中 M 为正交投影矩阵 (orthogonal projection matrix).

事实上approximation-based TD Learning 算法 (7.12) 的目的是最小化 projected Bellman error J_{PBE} 而非 J_E 和 J_{BE}. 此处仅考虑线性情形 $\hat{v}(w) = \Phi w$，Φ 的 range space 为所有可能的线性近似的集合. 此时正交投影矩阵 M 为:

$$M = \Phi(\Phi^TD\Phi)^{-1}\Phi^TD \in \mathbb{R}^{n\times n}. \tag{7.24}$$

由于 $\hat{v}(w)$ 和 $MT_\pi(\hat{v}(w))$ 均在 Φ 的 range space 中，因此能够找到 w 使得 $\min J_{PBE} = 0$. 同时有 (证明见附录**??**):

Theorem 7.1. 在线性情形 $\hat{v}(w^*) = \Phi w^*$ 下，估计值 $\hat{v}$ 与真实值 v_π 满足

$$\|\hat{v}(w^*) - v_\pi\|_D = \|\Phi w^* - v_\pi\|_D \leqslant \frac{1}{1-\gamma}\min_w \|\hat{v}(w) - v_\pi\|_D = \frac{1}{1-\gamma}\min_w \sqrt{J_E(w)}. \tag{7.25}$$

其中 γ 为 *discounted rate*.

Note 7.5. 虽然 *Theorem 7.1* 说明估计值与真实值差异的上界被 J_E 控制，但是实际这个 *upper bound* 很松，尤其是当 γ 很接近 *1* 时，因此主要具有理论意义.

7.4 Function Approximation based Sarsa

Function Approximation based Sarsa

Sarsa 与之前的算法唯一的不同就在于是在使用 $\hat{q}(s,a,w)$ 估计 action value $q(s,a)$ 而不是使用 $\hat{v}(s,w)$ 来估计 state value $v(s)$，因此非常自然地可以得到基于 function approximation 的 Sarsa 算法:

$$w_{t+1} = w_t + \alpha_t \left[r_{t+1} + \gamma \hat{q}(s_{t+1}, a_{t+1}, w_t) - \hat{q}(s_t, a_t, w_t) \right] \nabla_w \hat{q}(s_t, a_t, w_t). \tag{7.26}$$

式(7.26)实际上仍然在做 policy evaluation，将其与 policy improvement 结合就得到完整的 Sarsa 算法:

Algorithm 11 Function Approximation based Sarsa

1: **Aim:** Search a policy that can lead the agent to the target from an initial state-action pair (s_0, a_0).
2: **for** each episode **do**
3: **if** the current s_t is not the target state **then**
4: Take action a_t following $\pi_t(s_t)$, generate r_{t+1}, s_{t+1}, and then take action a_{t+1} following $\pi_t(s_{t+1})$
5: **Value update (parameter update):**

$$w_{t+1} = w_t + \alpha_t \left[r_{t+1} + \gamma \hat{q}(s_{t+1}, a_{t+1}, w_t) - \hat{q}(s_t, a_t, w_t) \right] \nabla_w \hat{q}(s_t, a_t, w_t)$$

6: **Policy update:**

$$\pi_{t+1}(a \mid s_t) = \begin{cases} 1 - \frac{\epsilon}{|A(s)|-1} & \text{if } a = \arg\max_a \hat{q}(s_t, a, w_{t+1}) \\ \frac{\epsilon}{|A(s)|} & \text{otherwise.} \end{cases}$$

7: **end if**
8: **end for**

7.5 Deep Q-Learning

Deep Q-Learning / Deep Q-Network

只需要将 Sarsa 中的 $\hat{q}(s_{t+1}, a_{t+1}, w_t)$ 替换为 $\max_{a \in \mathcal{A}(s_{t+1})} \hat{q}(s_{t+1}, a, w_t)$ 即可得到 approximation based Q-learning:

$$w_{t+1} = w_t + \alpha_t \left[r_{t+1} + \gamma \max_{a \in \mathcal{A}(s_{t+1})} \hat{q}(s_{t+1}, a, w_t) - \hat{q}(s_t, a_t, w_t) \right] \nabla_w \hat{q}(s_t, a_t, w_t) \tag{7.27}$$

Algorithm 12 Q-learning with Function Approximation (on-policy version)

1: **Initialization:** Initial parameter vector w_0. Initial policy π_0. Small $\epsilon > 0$.
2: **Aim:** Search a good policy that can lead the agent to the target from an initial state-action pair (s_0, a_0).
3: **for** each episode **do**
4: **if** the current s_t is not the target state **then**
5: Take action a_t following $\pi_t(s_t)$, and generate r_{t+1}, s_{t+1}.
6: **Value update (parameter update):**

$$w_{t+1} = w_t + \alpha_t \left[r_{t+1} + \gamma \max_{a \in A(s_{t+1})} \hat{q}(s_{t+1}, a, w_t) - \hat{q}(s_t, a_t, w_t) \right] \nabla_w \hat{q}(s_t, a_t, w_t)$$

7: **Policy update:** ▷ Since this is on-policy, so exploration needed

$$\pi_{t+1}(a \mid s_t) = \begin{cases} 1 - \frac{\epsilon}{|A(s_t)|-1} & \text{if } a = \arg\max_{a' \in A(s_t)} \hat{q}(s_t, a', w_{t+1}) \\ \frac{\epsilon}{|A(s_t)|} & \text{otherwise.} \end{cases}$$

8: **end if**
9: **end for**

Deep Q-Learning – Basic Idea

Deep Q-learning 也被称为Deep Q-network(DQN)，它是最早也是最成功的将深度学习引入强化学习的算法[a]. 其利用深度学习强大的函数逼近能力将深度学习作为非线性函数逼近器 (nonlinear function approximator) 拟合 q_π. 但有别于式 (7.27) 的是：由于神经网络强大的表达能力，我们不需要再底层地显式计算 $\hat{q}$.

Objective / Loss Function. DQN 目的是最小化如下目标函数/损失函数:

$$J(w) = \mathbb{E}\left[\left(R + \gamma \max_{a\in\mathcal{A}(S')} \hat{q}(S', a, w) - \hat{q}(S, A, w)\right)^2\right] \tag{7.28}$$

其中 (S, A, R, S') 为随机变量. 此 loss function 为 Bellman optimality error，因为根据 BOE 定义:

$$q(s, a) = \mathbb{E}\left[R_{t+1} + \gamma \max_{a\in\mathcal{A}(S_{t+1})} q(S_{t+1}, a)|S_t = s, A_t = a\right], \quad \forall s, a. \tag{7.29}$$

因此在期望意义下 $R + \gamma \max_{a\in\mathcal{A}(S')} \hat{q}(S', a, w) - \hat{q}(S, A, w)$ 应当为0.

Optimization. 最小化损失函数最直接的想法就是使用梯度下降 (GD). 在损失函数式(7.28)中参数 w 不仅存在于 $\hat{q}(S, A, w)$ 中，还存在于不好计算梯度的 $\gamma \max_{a\in\mathcal{A}(S')} \hat{q}(S', a, w)$ 中，因此此处将前半部分视作相对于 w 固定的整体，无需计算梯度，即定义:

$$y \triangleq R + \gamma \max_{a\in\mathcal{A}(S')} \hat{q}(S', a, w). \tag{7.30}$$

DQN Technique 1: Two Networks. Deep Q-learning 实际引入了两个不同的网络，其中一个网络即 $\hat{q}(s, a, w)$，用于逼近 q_π；第二个网络为了表示 $\hat{q}(s, a, w_T)$，将 $\hat{q}(s, a, w_T)$ 中的参数 w 固定方便优化时只针对第一个网络:

1. **main network:** $\hat{q}(s, a, w)$，用于逼近 q_π
2. **target network:** 用于表示 $\hat{q}(s, a, w_T)$，每隔一段时间就将 main network 的参数 w 赋值给 w_T，并持续一段时间内将 $\hat{q}(s, a, w_T)$ 固定、无需进行梯度下降，只针对 main network 进行优化.

这样损失函数就变为了:

$$J = \mathbb{E}\left[\left(R + \gamma \max_{a\in\mathcal{A}(S')} \hat{q}(S', a, w_T) - \hat{q}(S, A, w)\right)^2\right] \tag{7.31}$$

整个算法和训练的流程即:

$$\text{将 } w \text{ 赋值给 } w_T \to \text{固定 } w_T \text{ 只更新 } \hat{q}(S, A, w) \text{ 中的 } w \to \text{将 } w \text{ 赋值给 } w_T$$

DQN Technique 2: Experience Replay. 训练数据实际是智能体与环境交互产生的轨迹 (trajectory) 中得到的，例如一个 trajectory $S_t \to S_{t+1} \to S_{t+2}$ 可以得到有先后顺序的数据:

$$(S_0, A_0, R_1, S_1), (S_1, A_1, R_2, S_2), (S_2, A_2, R_3, S_3), \ldots \tag{7.32}$$

然而这些有先后顺序的数据具有强相关性，并且可能非常相似，因此这个 batch 计算出的梯度主要反映了某一段连续轨迹而不是整个经验分布，会导致梯度估计方差较大和学习不稳定. 因此实际中使用 experience replay，即数据打散后归入一个集合 $\mathcal{B} \triangleq \{(s, a, r, s')\}$，称之为 replay buffer，然后使用均匀分布进行随机抽取.

Note 7.6. *Tabular Q-learning* 不需要 *experience replay*，因为在表格形式中根本没有使用到分布的需要，是在不断求解所有 (s, a)的 *BOE*，而在 *DQN* 中，损失函数定义使用了期望，因此需要分布. 但也可以强制使用 *experience replay*，这样可以反复使用一次采样的经验，能够让数据 *(sample)* 使用地更高效.

[a] 虽然在 DQN 之前也有一些将深度学习结合强化学习的尝试，但是 DQN 是效果最好的，能够在一些游戏任务上达到人类玩家水准.

DQN Algorithm

Algorithm 13 Deep Q-learning (off-policy version)

Aim: Learn an optimal target network to approximate the optimal action values from the experience samples generated by a behavior policy π_b.
Store the experience samples generated by π_b in a replay buffer $\mathcal{B} = \{(s, a, r, s')\}$.
for each iteration **do**
    Uniformly draw a mini-batch of samples from $\mathcal{B}$.
    for each sample (s, a, r, s') **do**
        Calculate the target value as ▷ w_T is the parameter of the target network

$$y_T = r + \gamma \max_{a' \in A(s')} \hat{q}(s', a', w_T),$$

    end for
    Update the **main network** to minimize

$$(y_T - \hat{q}(s, a, w))^2$$

    using the mini-batch $\{(s, a, y_T)\}$.
    Set $w_T = w$ every C iterations.
end for

Note 7.7. *1.* 由于算法*13*是 *off-policy* 的，因此不需要进行 *policy update*

2. 为利用神经网络高效的"黑盒"性，因此不再使用之前基于梯度的 *policy update* 方式

3. DQN 原文中的算法是 *on-policy* 的，与此处略有不同